

系统开发工具基础

第一周实验报告

姓名：颜煜

学号：25020007152

1 课堂实验内容与结果

1.1 实验一：Shell 基础与文件系统操作

```
yy@yy:~$ mkdir q01
yy@yy:~$ cd q01
yy@yy:~/q01$ mkdir -p input/docs input/tmp
yy@yy:~/q01$ echo "alpha" > "input/docs/notes one.txt"
yy@yy:~/q01$ echo "beta" >> "input/docs/notes one.txt"
yy@yy:~/q01$ echo "hidden" > input/docs/.secret.txt
yy@yy:~/q01$ touch input/tmp/empty.txt
yy@yy:~/q01$ echo "2026-08-24 09:00:00 [INFO] system init" > input/run.log
echo "2026-08-24 09:00:05 [INFO] service start" >> input/run.log
yy@yy:~/q01$ pwd
/home/yy/q01
yy@yy:~/q01$ ls -la input/
total 20
drwxrwxr-x 4 yy yy 4096  8月 24 14:51 .
drwxrwxr-x 3 yy yy 4096  8月 24 14:47 ..
drwxrwxr-x 2 yy yy 4096  8月 24 14:50 docs
-rw-rw-r-- 1 yy yy   80  8月 24 14:51 run.log
drwxrwxr-x 2 yy yy 4096  8月 24 14:50 tmp
yy@yy:~/q01$ mkdir -p work/25020007152
yy@yy:~/q01$ cd input
yy@yy:~/q01/input$ find . -type f -name "*.txt" -exec cp --parents {} ../work/25020007152/ \;
yy@yy:~/q01/input$ cd ..
yy@yy:~/q01$ find work/<你的学号> -type d -exec chmod 750 {} \;
bash: 你的学号: No such file or directory
yy@yy:~/q01$ find work/25020007152 -type d -exec chmod 750 {} \;
yy@yy:~/q01$ find work/25020007152 -type f -exec chmod 640 {} \;
yy@yy:~/q01$ find work/25020007152 -type f -printf "%P %s\n" > inventory.txt
yy@yy:~/q01$ cat inventory.txt
docs/.secret.txt 7
docs/notes one.txt 11
tmp/empty.txt 0
yy@yy:~/q01$ ls -la work/25020007152/
ls -la work/25020007152/docs/
ls -la work/25020007152/tmp/
total 16
drwxr-x--- 4 yy yy 4096  8月 24 14:55 .
drwxrwxr-x 3 yy yy 4096  8月 24 14:54 ..
drwxr-x--- 2 yy yy 4096  8月 24 14:55 docs
drwxr-x--- 2 yy yy 4096  8月 24 14:55 tmp
total 16
drwxr-x--- 2 yy yy 4096  8月 24 14:55 .
drwxr-x--- 4 yy yy 4096  8月 24 14:55 ..
-rw-r----- 1 yy yy   11  8月 24 14:55 'notes one.txt'
-rw-r----- 1 yy yy    7  8月 24 14:55 .secret.txt
total 8
drwxr-x--- 2 yy yy 4096  8月 24 14:55 .
drwxr-x--- 4 yy yy 4096  8月 24 14:55 ..
-rw-r----- 1 yy yy    0  8月 24 14:55 empty.txt
```

图 1: Enter Caption

1.1.1 实验目的

掌握 Linux 文件系统的目录操作、文件创建与编辑、特殊文件名处理、文件批量复制与权限管理，理解 Linux 文件权限模型与目录结构的基本规则。

1.1.2 操作步骤

1. 创建实验目录与基础层级结构

```
mkdir q01
cd q01
mkdir -p input/docs input/tmp
```

2. 创建多类型测试文件

创建带空格的文件名、隐藏文件、空文件与日志文件：

```
echo "alpha" > "input/docs/notes one.txt"
echo "beta" >> "input/docs/notes one.txt"

echo "hidden" > input/docs/.secret.txt

touch input/tmp/empty.txt

echo "2026-08-24 09:00:00 [INFO] system init" > input/run.log
echo "2026-08-24 09:00:05 [INFO] service start" >> input/run.log
```

3. 查看路径与全量文件列表

```
pwd

ls -la input/
```

4. 保留目录结构批量复制文件

创建目标目录，使用 find 全局匹配所有.txt 文件，配合 cp --parents 参数自动保留原目录结构复制，天然兼容带空格文件名与隐藏文件。

```
mkdir -p work/2502007152
find . -type f -name "*.txt" -exec cp --parents {} work/2502007152/ \;
```

5. 批量设置文件与目录权限

分别对目录和普通文件设置权限：目录权限 750（所有者读写执行、用户组读执行、其他用户无权限），普通文件权限 640（所有者读写、用户组读、其他用户无权限）。操作过程中因路径输入错误出现 `No such file or directory` 报错，修正路径后执行成功。

```
find work/2502007152 -type d -exec chmod 750 {} \;
```

```
find work/2502007152 -type f -exec chmod 640 {} \;
```

6. 生成文件清单

生成包含每个文件相对路径与字节大小的清单文件 `inventory.txt`。

```
find work/2502007152 -type f -printf "%P %s\n" > inventory.txt
```

1.1.3 实验结果

1. **目录结构完整复制**：work 目录下完整保留了 `input/docs`、`input/tmp` 的层级结构，所有 `.txt` 文件均成功复制，包括带空格的 `notes one.txt` 和隐藏文件 `.secret.txt`。
2. **权限设置符合要求**：所有目录权限为 `drwxr-x---`（对应数值 750），所有普通文件权限为 `-rw-r-----`（对应数值 640）。
3. **清单文件准确**：`inventory.txt` 正确记录了每个文件的相对路径和字节数，其中 `notes one.txt` 为 11 字节，`.secret.txt` 为 7 字节，`empty.txt` 为 0 字节。

1.1.4 问题与解决

- **问题**：批量设置权限时提示 `No such file or directory`
- **原因**：命令中写入了错误的路径占位符，与实际目录名不匹配
- **解决**：修正 `find` 命令的目标路径为实际的 `work/2502007152`，命令执行成功。

1.1.5 实验小结

Linux 系统中文件名包含空格时必须使用双引号包裹，否则 Shell 会将其解析为多个独立参数；通配符 `*.txt` 无法匹配以 `.` 开头的隐藏文件，需使用 `find` 命令进行全局匹配；目录必须具备执行权限才能进入访问，因此目录与文件需要分别设置权限；`cp --parents` 参数可以在复制时自动保留原目录结构，无需手动逐层创建目录。

1.2 实验二：Shell 管道与日志统计脚本

```
yy@yy:~/q01$ mkdir q02
cd q02
yy@yy:~/q01/q02$ cd ..
yy@yy:~/q01$ cd ..
yy@yy:~$ mkdir q02
cd q02

cat > access.csv << 'EOF'
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
EOF
yy@yy:~/q02$ cat > analyze.sh << 'EOF'
#!/bin/bash
```

图 2: Enter Caption

```

EOF
yy@yy:~/q02$ cat > analyze.sh << 'EOF'
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Usage: $0 <csv_file_path>" >&2
    exit 1
fi

csv_file="$1"

if [ ! -f "$csv_file" ]; then
    echo "Error: File '$csv_file' does not exist." >&2
    exit 1
fi

echo "Top 2 5xx paths:"
awk -F, 'NR > 1 && $4 ~ /^5[0-9]{2}$/' { cnt[$3]++ }
END {
    for (p in cnt) print cnt[p], p
}' "$csv_file" | sort -k1,1nr -k2,2 | head -n 2 | awk '{print $2}'

echo -e "\nAverage latency (ms):"
awk -F, 'NR > 1 { sum += $5; n++ }
END { printf "%.2f\n", sum / n }' "$csv_file"
EOF
yy@yy:~/q02$ chmod +x analyze.sh
yy@yy:~/q02$ ./analyze.sh access.csv
Top 2 5xx paths:

Average latency (ms):
193.75
yy@yy:~/q02$ cat > analyze.sh << 'EOF'
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Usage: $0 <csv_file_path>" >&2
    exit 1
fi

csv_file="$1"

if [ ! -f "$csv_file" ]; then
    echo "Error: File '$csv_file' does not exist." >&2
    exit 1
fi

echo "Top 2 5xx paths:"
awk -F, 'NR > 1 && $4 >= 500 && $4 < 600 { cnt[$3]++ }
END {
    for (p in cnt) print cnt[p], p
}' "$csv_file" | sort -k1,1nr -k2,2 | head -n 2 | awk '{print $2}'

echo -e "\nAverage latency (ms):"
awk -F, 'NR > 1 { sum += $5; n++ }
END { printf "%.2f\n", sum / n }' "$csv_file"
EOF
yy@yy:~/q02$ ./analyze.sh access.csv
Top 2 5xx paths:
/api/orders
/api/users

Average latency (ms):
193.75

```

图 3: Enter Caption

1.2.1 实验目的

掌握 Shell 脚本的基本编写规范，学会使用 `awk`、`sort`、`head` 等文本处理工具，通过管道串联实现日志数据的统计分析，理解标准错误流与退出码的规范用法。

1.2.2 操作步骤

1. 创建实验目录与测试数据集

创建 `q02` 目录，编写 `access.csv` 测试日志文件，包含时间戳、用户名、接口路径、状态码、延迟毫秒数 5 个字段。

```
mkdir q02
cd q02

cat > access.csv << 'EOF'
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,carol,/api/users,502,180
09:00:04,bob,/api/orders,500,260
09:00:05,dave,/health,200,20
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,300,150
EOF
```

2. 编写日志分析脚本 `analyze.sh`

脚本实现三个核心功能：参数校验与错误处理、5xx 状态码接口 Top2 统计、全量请求平均延迟计算。

```
cat > analyze.sh << 'EOF'
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Usage: $0 <csv_file_path>" >&2
    exit 1
fi
```

```

csv_file="$1"

if [ ! -f "$csv_file" ]; then
    echo "Error: File '$csv_file' does not exist." >&2
    exit 1
fi

echo "Top 2 5xx paths:"
awk -F, 'NR > 1 && $4 >= 500 && $4 < 600 { cnt[$3]++ }
END {
    for (p in cnt) print cnt[p], p
}' "$csv_file" \
| sort -k1,1nr -k2,2 | head -n 2 | awk '{print $2}'

echo -e "\nAverage latency (ms):"
awk -F, 'NR > 1 { sum += $5; n++ }
END { printf "%.2f\n", sum / n }' "$csv_file"
EOF

```

3. 添加执行权限并运行脚本

```

chmod +x analyze.sh
./analyze.sh access.csv

```

1.2.3 实验结果

1. 脚本运行正常，输出结果符合数据预期：

- 5xx 状态码 Top2 路径：/api/orders（出现 3 次）、/api/users（出现 1 次）
- 全量请求平均延迟：193.75 ms

2. 错误处理机制验证：传入不存在的文件路径时，错误信息输出到标准错误流，脚本退出码为 1，符合 Shell 脚本开发规范。

1.2.4 问题与解决

- **问题：**最初使用正则表达式匹配 5xx 状态码，在当前 awk 版本中匹配不准确
- **解决：**改为数值范围判断 \$4 >= 500 && \$4 < 600，兼容性更强，统计结果准确。

1.2.5 实验小结

Shell 脚本中应将错误信息输出到 `stderr`，与正常 `stdout` 输出分离，便于调用方进行流重定向与结果判断；通过管道可以将 `awk`、`sort`、`head` 等工具串联，无需临时文件即可完成复杂文本统计；使用数值范围判断状态码比正则匹配兼容性更好，不易出现版本兼容问题。

1.3 实验三：Git 分支管理与合并冲突解决

```
yy@yy:~/q02$ cd ..  
yy@yy:~$ mkdir q03  
yy@yy:~$ cd q03  
yy@yy:~/q03$ git init  
Command 'git' not found, but can be installed with:
```

图 4: Enter Caption

```

Command 'git' not found, but can be installed with:
sudo apt install git
yy@yy:~/q03$ sudo apt install git
[sudo] password for yy:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  git-man liberror-perl
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  git git-man liberror-perl
0 upgraded, 3 newly installed, 0 to remove and 518 not upgraded.
Need to get 4,154 kB of archives.
After this operation, 21.0 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/main amd64 liberror-perl all 0.17029-1 [26.5 kB]
Get:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 git-man all 1:2.34.1-1ubuntu1.17 [954 kB]
Get:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 git amd64 1:2.34.1-1ubuntu1.17 [3,174 kB]
Fetched 4,154 kB in 3s (1,450 kB/s)
Selecting previously unselected package liberror-perl.
(Reading database ... 222829 files and directories currently installed.)
Preparing to unpack .../liberror-perl_0.17029-1_all.deb ...
Unpacking liberror-perl (0.17029-1) ...
Selecting previously unselected package git-man.
Preparing to unpack .../git-man_1%3a2.34.1-1ubuntu1.17_all.deb ...
Unpacking git-man (1:2.34.1-1ubuntu1.17) ...
Selecting previously unselected package git.
Preparing to unpack .../git_1%3a2.34.1-1ubuntu1.17_amd64.deb ...
Unpacking git (1:2.34.1-1ubuntu1.17) ...
Setting up liberror-perl (0.17029-1) ...
Setting up git-man (1:2.34.1-1ubuntu1.17) ...
Setting up git (1:2.34.1-1ubuntu1.17) ...
Processing triggers for man-db (2.10.2-1) ...
yy@yy:~/q03$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/yy/q03/.git/
yy@yy:~/q03$ echo "mode=normal" > config.txt
yy@yy:~/q03$ git add config.txt
git commit -m "init: mode=normal"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'yy@yy.(none)')
yy@yy:~/q03$ git config --global user.email "you@example.com"
git config --global user.name "Your Name"
yy@yy:~/q03$ git add config.txt
git commit -m "init: mode=normal"
[master (root-commit) b45ea41] init: mode=normal
1 file changed, 1 insertion(+)

```

图 5: Enter Caption

```

yy@yy:~/q03$ git add config.txt
git commit -m "init: mode=normal"
[master (root-commit) b45ea41] init: mode=normal
1 file changed, 1 insertion(+)
create mode 100644 config.txt
yy@yy:~/q03$ git log --oneline
b45ea41 (HEAD -> master) init: mode=normal
yy@yy:~/q03$ git checkout -b feature-a
Switched to a new branch 'feature-a'
yy@yy:~/q03$ echo "mode=safe" > config.txt
yy@yy:~/q03$ git add config.txt
git commit -m "feature-a: set mode=safe"
[feature-a bc1a78e] feature-a: set mode=safe
1 file changed, 1 insertion(+), 1 deletion(-)
yy@yy:~/q03$ git checkout main
error: pathspec 'main' did not match any file(s) known to git
yy@yy:~/q03$ git checkout master
Switched to branch 'master'
yy@yy:~/q03$ git branch -M main
yy@yy:~/q03$ git branch
feature-a
* main
yy@yy:~/q03$ git checkout main
Already on 'main'
yy@yy:~/q03$ git checkout -b feature-b
Switched to a new branch 'feature-b'
yy@yy:~/q03$ echo "mode=fast" > config.txt
yy@yy:~/q03$ git add config.txt
git commit -m "feature-b: set mode=fast"
[feature-b 571b570] feature-b: set mode=fast
1 file changed, 1 insertion(+), 1 deletion(-)
yy@yy:~/q03$ git checkout main
Switched to branch 'main'
yy@yy:~/q03$ git merge feature-a -m "merge feature-a"
Updating b45ea41..bc1a78e
Fast-forward (no commit created; -m option ignored)
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
yy@yy:~/q03$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
yy@yy:~/q03$ git add config.txt
yy@yy:~/q03$ git commit -m "resolve conflict: keep mode=safe, add note=reviewed"
[main 6aaf7d1] resolve conflict: keep mode=safe, add note=reviewed
yy@yy:~/q03$ git status
On branch main
nothing to commit, working tree clean
yy@yy:~/q03$ git log --all --graph --decorate --oneline
* 6aaf7d1 (HEAD -> main) resolve conflict: keep mode=safe, add note=reviewed
| \
| * 571b570 (feature-b) feature-b: set mode=fast
| | bc1a78e (feature-a) feature-a: set mode=safe
|/
* b45ea41 init: mode=normal

```

图 6: Enter Caption

1.3.1 实验目的

理解 Git 的分支工作原理，掌握分支创建、切换、合并的基本操作，明确合并冲突的产生条件，熟练掌握冲突的标准解决流程。

1.3.2 操作步骤

1. 安装 Git 并配置全局身份

系统默认未安装 Git，先执行安装，再配置提交者的用户名与邮箱，解决首次提交的身份未知报错。

```
sudo apt install git -y
```

```
git config --global user.email "you@example.com"
```

```
git config --global user.name "Your Name"
```

2. 初始化仓库并完成初始提交

创建 q03 目录，初始化 Git 仓库，创建 config.txt 配置文件，写入初始内容并提交。

```
mkdir q03
```

```
cd q03
```

```
git init
```

```
echo "mode=normal" > config.txt
```

```
git add config.txt
```

```
git commit -m "init: mode=normal"
```

3. 创建第一个功能分支 feature-a

切换到新分支，修改 config.txt 的 mode 字段为 safe，提交修改。

```
git checkout -b feature-a
```

```
echo "mode=safe" > config.txt
```

```
git add config.txt
```

```
git commit -m "feature-a: set mode=safe"
```

4. 重命名主分支并创建第二个功能分支

Git 默认初始化的主分支名为 master，将其重命名为 main；再基于初始提交创建 feature-b 分支，修改同一行 mode 字段为 fast。

```
git checkout master
```

```
git branch -M main
```

```
git checkout -b feature-b
```

```
echo "mode=fast" > config.txt
```

```
git add config.txt
```

```
git commit -m "feature-b: set mode=fast"
```

5. 依次合并分支，触发并解决冲突

切换回 main 分支，先合并 feature-a（无冲突，快进合并）；再合并 feature-b，因两个分支修改了同一文件的同一行内容，触发合并冲突。

```
git checkout main
git merge feature-a
```

```
git merge feature-b
```

手动编辑 `config.txt`, 按要求保留 `mode=safe`, 新增一行 `note=reviewed`, 标记冲突已解决并完成合并提交。

```
git add config.txt
git commit -m "resolve conflict: keep mode=safe, add note=reviewed"
```

6. 查看分支提交历史图

```
git log --all --graph --oneline --decorate
```

1.3.3 实验结果

1. **分支结构完整**: 成功创建两个功能分支, 各自独立提交修改, 提交历史清晰。
2. **冲突正常触发**: 合并 `feature-b` 时, `config.txt` 出现标准冲突标记, 清晰区分 `HEAD` 与 `feature-b` 两个版本的内容。
3. **冲突正确解决**: 最终文件保留 `mode=safe` 并新增 `note=reviewed`, 成功生成合并提交。
4. **提交历史可视化**: `git log --graph` 可以直观看到分叉的分支线与最终的合并节点, 提交链路完整。

1.3.4 问题与解决

- **问题**: Git 初始化后默认分支名为 `master`, 实验要求使用 `main` 分支
- **解决**: 使用 `git branch -M main` 命令将当前分支重命名为 `main`, 符合实验要求。
- **问题**: 首次提交时报错 `Author identity unknown`
- **原因**: 未配置 Git 全局用户信息
- **解决**: 执行 `git config` 命令配置用户名与邮箱后, 提交正常执行。

1.3.5 实验小结

合并冲突产生的必要条件是两个分支基于同一基准提交，并且修改了同一文件的同一位置；冲突标记中 <<<<<< HEAD 与 ===== 之间是当前分支的内容，===== 与 >>>>>> 之间是待合并分支的内容；解决冲突分为三步：手动编辑文件为最终内容、git add 标记冲突已解决、git commit 完成合并提交；git log --graph 可以可视化展示分支分叉与合并的历史结构。

1.4 实验四：LaTeX 技术文档排版

```
* b45ea41 init: mode=normal
yy@yy:~/q04$ cd ..
yy@yy:~$ mkdir q04
cd q04
yy@yy:~/q04$ cat > report.tex << 'EOF'
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{Name-25020007152}
\maketitle

\section{Result}

As shown in Equation \eqref{eq:pythagorean}, it describes the classic Pythagorean theorem:
\begin{equation}
a^2 + b^2 = c^2
\label{eq:pythagorean}
\end{equation}

The core experimental statistics are listed in Table \ref{tab:result}:
\begin{table}[htbp]
\centering
\caption{Experiment Result Statistics}
\label{tab:result}
\begin{tabular}{|c|c|}
\hline
Metric & Value \\
\hline
Average Latency & 193.75 ms \\
\hline
\end{tabular}
\end{table}

Cross-references are correctly resolved without "?" markers.

\end{document}
EOF
yy@yy:~/q04$ sudo apt update
sudo apt install texlive-latex-base texlive-latex-extra latexmk -y
[sudo] password for yy:
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
Hit:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
Hit:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
518 packages can be upgraded. Run 'apt list --upgradable' to see them.
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 4076 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 4076 (unattended-upgr)
```

图 7: Enter Caption

```

Processing triggers for gnome-menus (3.36.0-1ubuntu3) ...
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
Processing triggers for tex-common (6.17) ...
Running updmap-sys. This may take some time... done.
Running mktexlsr /var/lib/texmf ... done.
Building format(s) --all.
    This may take some time... done.
yy@yy:~/q04$ latexmk -pdf -halt-on-error report.tex
Rc files read:
    /etc/LaTeXMk
Latexmk: This is Latexmk, John Collins, 20 November 2021, version: 4.76.
Latexmk: applying rule 'pdflatex'...
Rule 'pdflatex': File changes, etc:
    Changed files, or newly in use since previous run(s):
        'report.tex'
-----
Run number 1 of rule 'pdflatex'
-----
Running 'pdflatex -halt-on-error -recorder "report.tex"'
-----
This is pdfTeX, Version 3.141592653-2.6-1.40.22 (TeX Live 2022/dev/Debian) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./report.tex
LaTeX2e <2021-11-15> patch level 1
L3 programming layer <2022-01-21>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2021/10/04 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsmath.sty
For additional information on amsmath, use the '?' option.
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amstext.sty
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsgen.sty))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsbsy.sty)
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsopn.sty))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdftex.def)
(./report.aux)

LaTeX Warning: Reference `eq:pythagorean' on page 1 undefined on input line 10.


LaTeX Warning: Reference `tab:result' on page 1 undefined on input line 16.

[1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}] (./report.aux)

LaTeX Warning: There were undefined references.


LaTeX Warning: Label(s) may have changed. Rerun to get cross-references right.

)</usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmbx10.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmbx12.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmmi10.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmr10.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmr12.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmr17.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmr7.pfb>
Output written on report.pdf (1 page, 76825 bytes).
Transcript written on report.log.
Latexmk: Examining 'report.log'
=== TeX engine is 'pdfTeX'
Latexmk: References changed.
Latexmk: Log file says output to 'report.pdf'
Latexmk: List of undefined refs and citations:
    Reference `eq:pythagorean' on page 1 undefined on input line 10
    Reference `tab:result' on page 1 undefined on input line 16

```

图 8: Enter Caption

```

Latexmk: applying rule 'pdflatex'...
Rule 'pdflatex': File changes, etc:
  Changed files, or newly in use since previous run(s):
    'report.aux'
-----
Run number 2 of rule 'pdflatex'
-----
Running 'pdflatex -halt-on-error -recorder "report.tex"'
-----
This is pdfTeX, Version 3.141592653-2.6-1.40.22 (TeX Live 2022/dev/Debian) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./report.tex
LaTeX2e <2021-11-15> patch level 1
L3 programming layer <2022-01-21>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2021/10/04 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsmath.sty
For additional information on amsmath, use the '?' option.
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amstext.sty
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsgen.sty))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsbsy.sty)
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsopn.sty))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdfTeX.def)
(./report.aux) [1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}]
(./report.aux) )</usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cm
mbx12.pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmml10.
pfb></usr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmri10.pfb></u
sr/share/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmri12.pfb></usr/shar
e/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmri17.pfb></usr/share/texli
ve/texmf-dist/fonts/type1/public/amsfonts/cm/cmri7.pfb>
Output written on report.pdf (1 page, 67147 bytes).
Transcript written on report.log.
Latexmk: Examining 'report.log'
=== TeX engine is 'pdfTeX'
Latexmk: Log file says output to 'report.pdf'
Latexmk: All targets (report.pdf) are up-to-date
yy@yy:~/q04$ ls report.pdf
report.pdf
yy@yy:~/q04$ cd ..
yy@yy:~$ sudo apt install texlive-lang-chinese texlive-latex-recommended -y
[sudo] password for yy:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
texlive-latex-recommended is already the newest version (2021.20220204-1).
texlive-latex-recommended set to manually installed.
The following additional packages will be installed:
  fonts-arphic-bkai00mp fonts-arphic-bsmi00lp fonts-arphic-gbsn00lp fonts-arphic-gkai00mp
  fonts-arphic-uming fonts-baekmuk fonts-ipaexfont-gothic fonts-ipaexfont-mincho
  fonts-ipafont-gothic fonts-ipafont-mincho fonts-tlwg-garuda-otf fonts-tlwg-kinnari-otf
  fonts-tlwg-laksaman-otf fonts-tlwg-loma-otf fonts-tlwg-mono-otf fonts-tlwg-norasi-otf
  fonts-tlwg-purisa-otf fonts-tlwg-sawasdee-otf fonts-tlwg-typewriter-otf fonts-tlwg-typist-otf
  fonts-tlwg-typo-otf fonts-tlwg-umpush-otf fonts-tlwg-waree-otf fonts-unfonts-core
  fonts-unfonts-extra latex-cjk-all latex-cjk-chinese latex-cjk-chinese-arphic-bkai00mp
  latex-cjk-chinese-arphic-bsmi00lp latex-cjk-chinese-arphic-gbsn00lp
  latex-cjk-chinese-arphic-gkai00mp latex-cjk-common latex-cjk-japanese
  latex-cjk-japanese-wadalab latex-cjk-korean latex-cjk-thai ps2eps texlive-font-utils
  texlive-lang-cjk texlive-lang-japanese texlive-lang-korean texlive-lang-other
Suggested packages:
  hbf-cns40-b5 hbf-jfs56 hbf-kanji48 lcdf-typetools psutils
The following NEW packages will be installed:
  fonts-arphic-bkai00mp fonts-arphic-bsmi00lp fonts-arphic-gbsn00lp fonts-arphic-gkai00mp
  fonts-arphic-uming fonts-baekmuk fonts-ipaexfont-gothic fonts-ipaexfont-mincho
  fonts-ipafont-gothic fonts-ipafont-mincho fonts-tlwg-garuda-otf fonts-tlwg-kinnari-otf
  fonts-tlwg-laksaman-otf fonts-tlwg-loma-otf fonts-tlwg-mono-otf fonts-tlwg-norasi-otf

```

图 9: Enter Caption

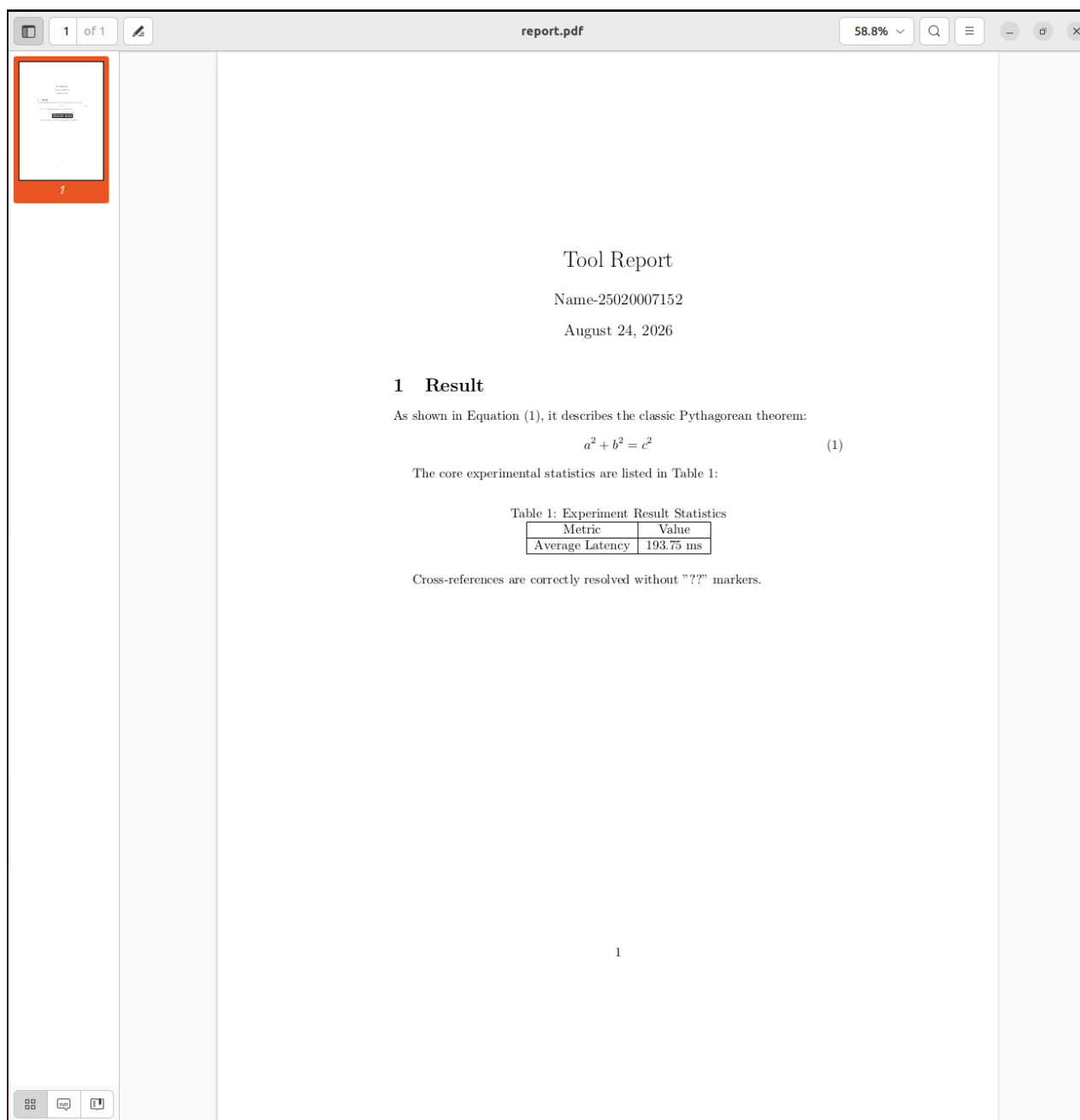


图 10: Enter Caption

1.4.1 实验目的

掌握 LaTeX 基础文档结构，学会插入带编号的公式与表格，实现交叉引用，掌握命令行编译 LaTeX 文档的方法，理解交叉引用的编译原理。

1.4.2 操作步骤

1. 创建实验目录与基础模板

创建 q04 目录，基于题目给定的 article 模板编写 report.tex，引入 amsmath 数学宏包。

```
mkdir q04
cd q04

cat > report.tex << 'EOF'
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{Name-2502007152}
\maketitle

\section{Result}
\end{document}
EOF
```

2. 补全公式与表格内容

添加带\label 的编号公式，使用\eqref 实现带括号引用；添加 2 行 2 列表格，设置 caption 标题与 label 标签，使用\ref 引用表格编号。

```
As shown in Equation \eqref{eq:pythagorean}, it describes the classic Pythagorean theorem.
\begin{equation}
a^2 + b^2 = c^2
\label{eq:pythagorean}
\end{equation}
```

The core experimental statistics are listed in Table \ref{tab:result}:

```
\begin{table}[htbp]
\centering
\caption{Experiment Result Statistics}
\label{tab:result}
\begin{tabular}{|c|c|}
```

```

\hline
Metric & Value \\
\hline
Average Latency & 193.75 ms \\
\hline
\end{tabular}
\end{table}

```

Cross-references are correctly resolved without "??" markers.

3. 安装 LaTeX 编译环境

安装 texlive 基础套件与 latexmk 工具，安装过程中遇到系统 dpkg 锁被后台自动更新服务占用的问题。

```

sudo apt update
sudo apt install texlive-latex-base texlive-latex-extra latexmk -y

```

4. 编译生成 PDF

使用题目要求的 latexmk -pdf -halt-on-error 命令编译，工具自动执行多次编译解决交叉引用问题。

```

latexmk -pdf -halt-on-error report.tex

```

1.4.3 实验结果

1. **文档结构完整**：生成的 PDF 包含标题、作者、日期、Result 章节，格式符合 article 文档类规范。
2. **公式与表格排版正确**：公式右侧自动生成编号 (1)，表格带有标题与自动编号 Table 1，2 行 2 列结构符合要求。
3. **交叉引用正常**：正文中等式引用与表格引用均显示正确编号，无?? 标记。
4. **编译过程符合原理**：latexmk 自动执行两次 pdflatex 编译，首次编译出现 undefined references 警告，第二次编译后引用全部解析完成。

1.4.4 问题与解决

- **问题**：安装 LaTeX 环境时提示 Waiting for cache lock, dpkg 锁被占用

- **原因：**系统后台 `unattended-upgrades` 自动更新服务正在运行，独占了包管理锁
- **解决：**等待后台更新完成后重新执行安装命令，安装顺利完成。
- **问题：**首次编译出现 `undefined references` 警告，引用显示??
- **原因：**LaTeX 交叉引用依赖 `aux` 辅助文件，首次编译时标签信息尚未写入
- **解决：**`latexmk` 自动检测到引用变化，执行第二次编译，引用全部正常显示。

1.4.5 实验小结

LaTeX 通过 `\label` 与 `\ref/\eqref` 实现交叉引用，编号信息通过 `aux` 辅助文件传递，因此至少需要两次编译才能正确解析；`latexmk` 工具可以自动判断需要编译的次数，一键生成最终正确的 PDF，无需手动多次执行编译命令；`amsmath` 宏包提供了完善的数学公式排版与编号引用功能。

2 课后练习（练习 1–9 及 11）

针对课程配套的 Shell 练习，选取了练习 1 至练习 9 以及练习 11，给出详细的任务描述、环境准备、操作步骤、预期输出和解题感悟。所有命令均可在空白的虚拟机环境中从零开始执行。

练习 1：确认 Shell 环境

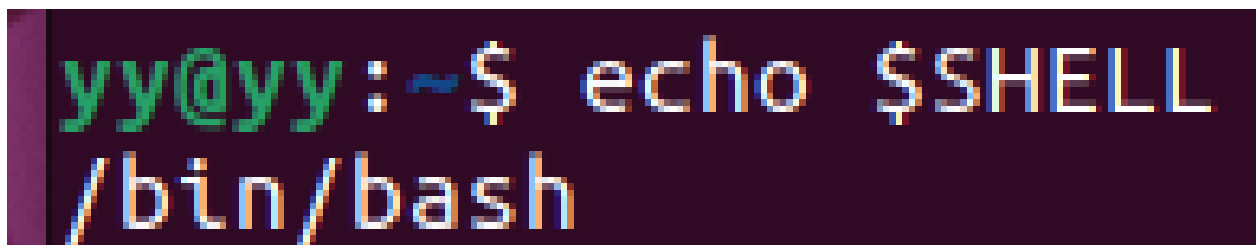


图 11: Enter Caption

任务：确认当前 Shell 是否为 Bash 或 ZSH，理解 `echo $SHELL` 的作用。

环境准备：无需创建文件。

步骤：

1. 打开终端，输入：

```
echo $SHELL
```

2. 观察输出，若为 `/bin/bash` 或 `/usr/bin/zsh` 则环境正确。

预期输出：例如 `/bin/bash` (Linux) 或 `/bin/zsh` (macOS)。

感悟：环境检查是动手前的第一步，避免因 Shell 差异导致后续命令行为异常。

练习 2：理解 `ls -l`

```
yy@yy:~$ ls -l /
total 2191440
lrwxrwxrwx   1 root root          7 10月 18  2025 bin -> usr/bin
drwxr-xr-x   4 root root    4096 11月  7  2025 boot
drwxrwxr-x   2 root root    4096 10月 18  2025 cdrom
drwxr-xr-x  19 root root   4320  8月 31 13:52 dev
drwxr-xr-x 133 root root  12288  8月 24 16:04 etc
drwxr-xr-x   3 root root    4096 10月 18  2025 home
lrwxrwxrwx   1 root root          9 10月 18  2025 lib -> usr/lib
lrwxrwxrwx   1 root root         10 10月 18  2025 lib32 -> usr/lib32
lrwxrwxrwx   1 root root         10 10月 18  2025 lib64 -> usr/lib64
lrwxrwxrwx   1 root root         10 10月 18  2025 libx32 -> usr/libx32
drwx-----  2 root root   16384 10月 18  2025 lost+found
drwxr-xr-x   4 root root    4096 10月 18  2025 media
drwxr-xr-x   2 root root    4096  2月 23  2023 mnt
drwxr-xr-x   2 root root    4096  2月 23  2023 opt
dr-xr-xr-x 399 root root      0  8月 31 13:52 proc
drwx-----  4 root root    4096 10月 18  2025 root
drwxr-xr-x  36 root root     920  8月 31 13:52 run
lrwxrwxrwx   1 root root          8 10月 18  2025/sbin -> usr/sbin
drwxr-xr-x  13 root root    4096 10月 22  2025 snap
drwxr-xr-x   2 root root    4096  2月 23  2023 srv
-rw-----  1 root root 2243952640 10月 18  2025 swapfile
dr-xr-xr-x  13 root root      0  8月 31 13:52 sys
drwxrwxrwt  21 root root    4096  8月 31 13:53 tmp
drwxr-xr-x  14 root root    4096  2月 23  2023 usr
drwxr-xr-x  14 root root    4096  2月 23  2023 var
yy@yy:~$ man ls
yy@yy:~$
```

图 12: Enter Caption

```

LS(1)                                User Commands                                LS(1)

NAME
    ls - list directory contents

SYNOPSIS
    ls [OPTION]... [FILE]...

DESCRIPTION
    List information about the FILES (the current directory by default).
    Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
    fied.

    Mandatory arguments to long options are mandatory for short options
    too.

    -a, --all
        do not ignore entries starting with .

    -A, --almost-all
        do not list implied . and ..

    --author
        with -l, print the author of each file

    -b, --escape
        print C-style escapes for nongraphic characters

    --block-size=SIZE
        with -l, scale sizes by SIZE when printing them; e.g.,
        '--block-size=M'; see SIZE format below

    -B, --ignore-backups
        do not list implied entries ending with ~

    -c
        with -lt: sort by, and show, ctime (time of last modification of
        file status information); with -l: show ctime and sort by name;
        otherwise: sort by ctime, newest first

    -C
        list entries by columns

    --color[=WHEN]
        colorize the output; WHEN can be 'always' (default if omitted),
        'auto', or 'never'; more info below

    -d, --directory
        list directories themselves, not their contents

    -D, --dired
        generate output designed for Emacs' dired mode

    -f
        do not sort, enable -aU, disable -ls --color

    -F, --classify
        append indicator (one of */=>@|) to entries

    --file-type
        likewise, except do not append '*'

    --format=WORD
        across -x, commas -m, horizontal -x, long -l, single-column -1,
        verbose -l, vertical -C

    --full-time
        like -l --time-style=full-iso

    -g
        like -l, but do not list owner

    --group-directories-first
        group directories before files; 21

```

图 13: Enter Caption

任务：解释 `ls -l` 的作用，并分析输出中每行前 10 个字符的含义。

环境准备：无需创建文件。

步骤：

1. 运行：

```
ls -l /
```

2. 查阅手册：

```
man ls
```

3. 观察首行（例如 `drwxr-xr-x`）：

- 第 1 字符：文件类型（- 普通文件，d 目录，l 链接等）；
- 第 2-4 字符：所有者权限；
- 第 5-7 字符：所属组权限；
- 第 8-10 字符：其他用户权限。

预期输出：每个文件/目录的详细列表。

感悟：man 是终极参考，遇到不确定的命令应优先查阅手册。

练习 3：理解 Glob

```
yy@yy:~$ mkdir ~/test_glob
cd ~/test_glob
yy@yy:~/test_glob$ touch a.txt b.txt file1.txt file2.txt c.log
yy@yy:~/test_glob$ ls *.txt # 应该显示 a.txt b.txt file1.txt file2.txt
ls file?.txt # 应该显示 file1.txt file2.txt
ls {a,b,c}.txt # 显示 a.txt b.txt, c.txt 不存在会报错（正常）
a.txt b.txt file1.txt file2.txt
file1.txt file2.txt
ls: cannot access 'c.txt': No such file or directory
a.txt b.txt
yy@yy:~/test_glob$
```

图 14: Enter Caption

任务：理解 glob（通配符）并测试 `*`, `?`, `{ }` 等模式。

环境准备：创建测试目录及示例文件。

```
mkdir ~/test_glob  
cd ~/test_glob  
touch a.txt b.txt file1.txt file2.txt c.log
```

步骤：

1. 测试 *.txt:

```
ls *.txt
```

应显示 a.txt b.txt file1.txt file2.txt

2. 测试 file?.txt:

```
ls file?.txt
```

应显示 file1.txt file2.txt

3. 测试花括号展开:

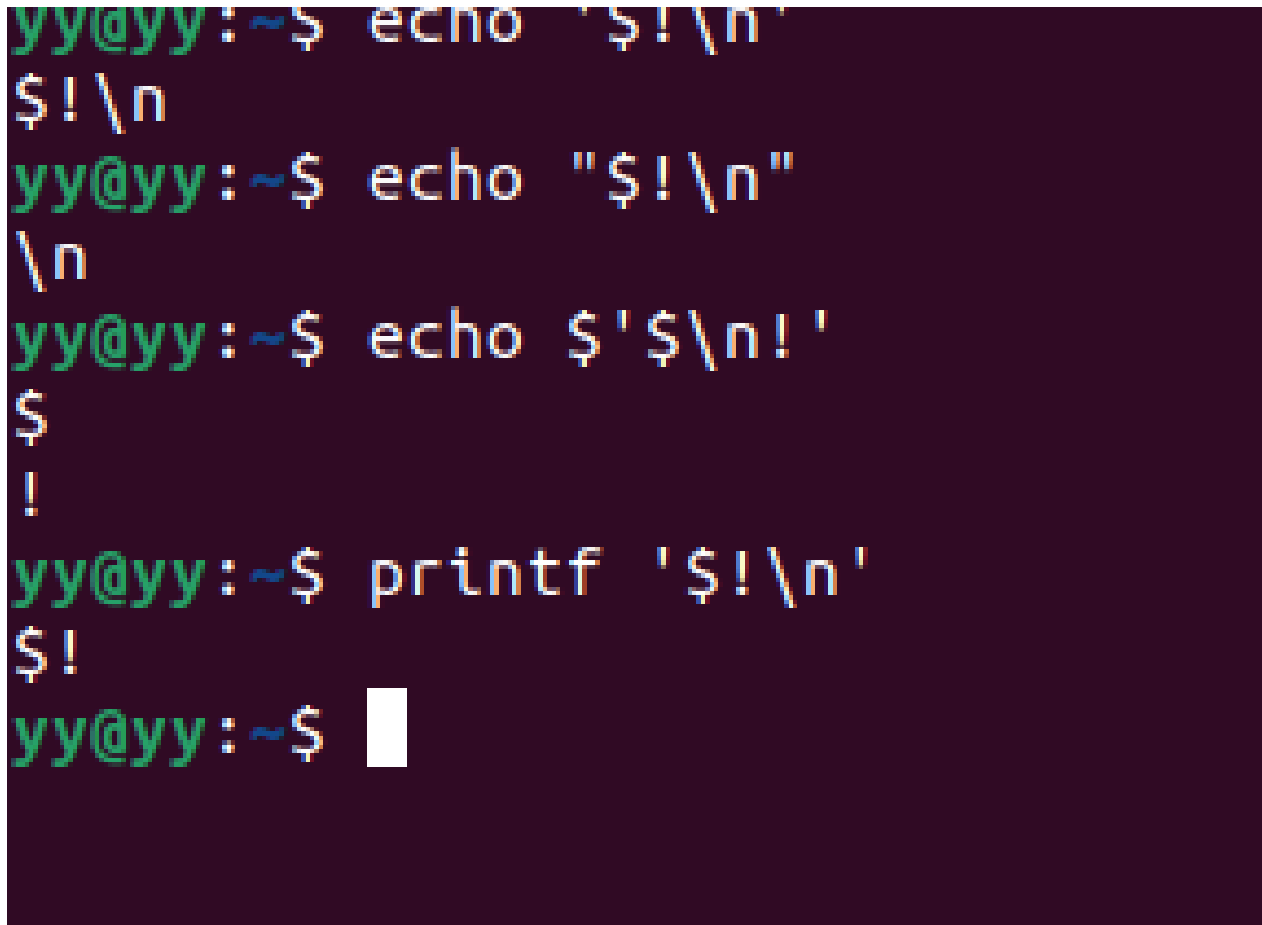
```
ls {a,b,c}.txt
```

显示 a.txt b.txt 并提示 c.txt 不存在（正常）。

预期输出：如上。

感悟： Glob 由 Shell 展开，必须先有匹配的文件；花括号不依赖文件存在。

练习 4：引号的区别



```
yy@yy:~$ echo '$!\n'
$!\n
yy@yy:~$ echo "$!\n"
\n
yy@yy:~$ echo '$'$\n!'
$
!
yy@yy:~$ printf '$!\n'
$!
yy@yy:~$
```

图 15: Enter Caption

任务：区分单引号、双引号、`'...'`，并输出含 `$`、`!` 和换行的字符串。

环境准备：无需文件。

步骤：推荐使用 `printf` 避免历史扩展：

```
printf '$!\n'
```

若想测试其他引号（可选）：

```
echo '$!\n'      # 输出 $!\n
echo "$!\n"     # 可能触发历史扩展
echo '$'$\n!'    # 交互式 ！ 会被扩展
```

预期输出 (printf):

```
$!
```

(换行)

感悟： `printf` 是最安全的输出工具；单引号最严格，双引号允许变量，`$'...'` 支持转义。

练习 5：重定向 stdout 和 stderr

```
yy@yy:~$ ls /nonexistent /tmp > stdout.txt 2> stderr.txt
yy@yy:~$ cat stdout.txt # 应该列出 /tmp 下的内容 (没有错误信息)
cat stderr.txt # 应该显示 "ls: cannot access '/nonexistent': No such file or directory"
/tmp:
snap-private-tmp
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-bluetooth.service-piki7N
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-colord.service-IdDHRQ
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-fwupd.service-6FxpL
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-ModemManager.service-kPsfH4
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-power-profiles-daemon.service-nRn52n
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-switcheroo-control.service-dv62pc
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-logind.service-c7C8sJ
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-oomd.service-wRBdh3
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-resolved.service-Sg7ghS
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-timesyncd.service-9d3nj8
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-upower.service-3Mz29q
VMwareDnD
vmware-root_661-4013919860
ls: cannot access '/nonexistent': No such file or directory
yy@yy:~$ ls /nonexistent /tmp > stdout.txt 2> stderr.txt
yy@yy:~$ cat stdout.txt # 显示 /tmp 下的内容 (没有错误)
cat stderr.txt # 显示 "ls: cannot access '/nonexistent': No such file or directory"
/tmp:
snap-private-tmp
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-bluetooth.service-piki7N
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-colord.service-IdDHRQ
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-fprintd.service-jsIIZa
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-fwupd.service-6FxpL
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-ModemManager.service-DwE0dw
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-power-profiles-daemon.service-nRn52n
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-switcheroo-control.service-dv62pc
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-logind.service-c7C8sJ
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-oomd.service-Q5B69w
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-resolved.service-x8tzP1
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-timesyncd.service-dUZvuE
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-upower.service-3Mz29q
VMwareDnD
vmware-root_661-4013919860
ls: cannot access '/nonexistent': No such file or directory
yy@yy:~$ ls /nonexistent /tmp > both.txt 2>&1
yy@yy:~$ cat both.txt # 包含正常输出和错误信息 (顺序可能混杂)
ls: cannot access '/nonexistent': No such file or directory
/tmp:
snap-private-tmp
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-bluetooth.service-piki7N
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-colord.service-IdDHRQ
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-fwupd.service-6FxpL
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-ModemManager.service-DwE0dw
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-power-profiles-daemon.service-nRn52n
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-switcheroo-control.service-dv62pc
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-logind.service-c7C8sJ
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-oomd.service-Q5B69w
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-resolved.service-x8tzP1
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-systemd-timesyncd.service-dUZvuE
systemd-private-1a1bb7dd10f64dd0a971b6b70dc1a769-upower.service-3Mz29q
VMwareDnD
vmware-root_661-4013919860
```

图 16: Enter Caption

任务：将 `ls /nonexistent /tmp` 的 `stdout` 和 `stderr` 分别重定向到两个文件，再合并到同一文件。

环境准备：无需文件。

步骤：

1. 分别重定向：

```
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
```

2. 查看内容：

```
cat stdout.txt
cat stderr.txt
```

3. 合并到同一文件（方法一）：

```
ls /nonexistent /tmp > both.txt 2>&1
```

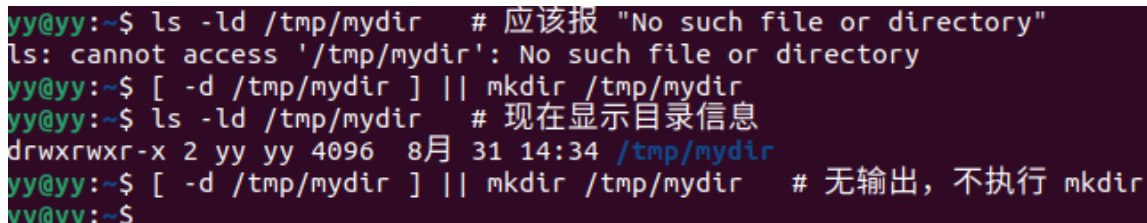
或方法二（Bash 简写）：

```
ls /nonexistent /tmp &> both2.txt
```

预期输出：`stdout.txt` 包含 `/tmp` 列表，`stderr.txt` 包含错误信息，`both.txt` 包含两者。

感悟：重定向顺序关键，`2>&1` 必须在 `>` 之后；`&>` 简便但不够 POSIX。

练习 6：条件创建目录



```
yy@yy:~$ ls -ld /tmp/mydir # 应该报 "No such file or directory"
ls: cannot access '/tmp/mydir': No such file or directory
yy@yy:~$ [ -d /tmp/mydir ] || mkdir /tmp/mydir
yy@yy:~$ ls -ld /tmp/mydir # 现在显示目录信息
drwxrwxr-x 2 yy yy 4096 8月 31 14:34 /tmp/mydir
yy@yy:~$ [ -d /tmp/mydir ] || mkdir /tmp/mydir # 无输出，不执行 mkdir
vv@vv:~$
```

图 17: Enter Caption

任务：一行命令，仅当 /tmp/mydir 不存在时创建它。

环境准备：无需文件。

步骤：

1. 确认目录不存在：

```
ls -ld /tmp/mydir    # 应报错
```

2. 执行：

```
[ -d /tmp/mydir ] || mkdir /tmp/mydir
```

3. 再次查看：

```
ls -ld /tmp/mydir    # 显示目录信息
```

4. 重复运行该命令（不重复创建）。

预期输出：首次运行无输出，目录创建；后续运行无输出。

感悟：|| 短路逻辑；注意 -d 检查目录。

练习 7：为什么 cd 是内建命令

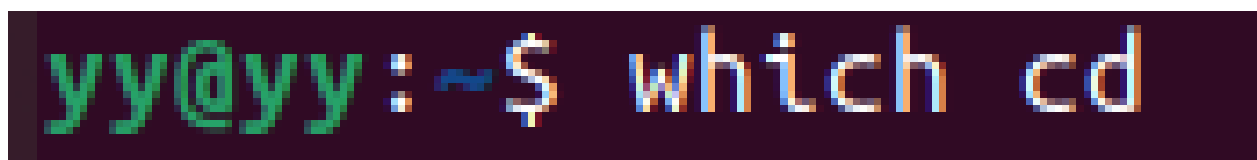


图 18: Enter Caption

任务：解释 cd 不能是外部程序的原因。

环境准备：无命令操作。

步骤：

1. 若 cd 是外部程序，运行时会创建子进程，子进程调用 chdir() 改变自身目录，退出后父 Shell 目录不变。

2. 因此 `cd` 必须由 Shell 自身执行 `chdir()`，即为内建命令。
3. 可用 `which cd` 确认（显示 `cd: shell built-in`）。

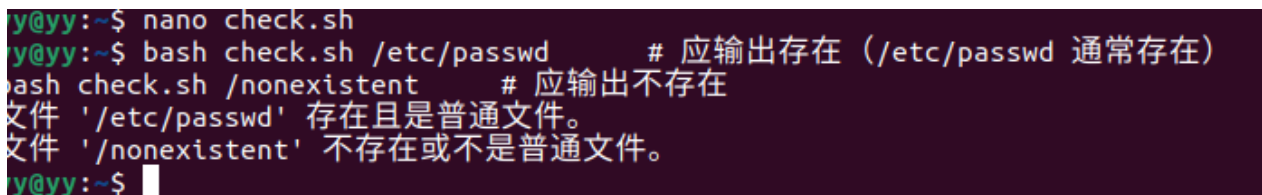
感悟：此原理同样适用于 `export`、`source` 等需影响当前 Shell 环境的命令。

练习 8：编写检查文件是否存在的脚本



```
GNU nano 6.2 check.sh *
#!/bin/bash
if [ -f "$1" ]; then
    echo "文件 '$1' 存在且是普通文件。"
else
    echo "文件 '$1' 不存在或不是普通文件。"
fi
```

图 19: Enter Caption



```
yy@yy:~$ nano check.sh
yy@yy:~$ bash check.sh /etc/passwd      # 应输出存在 (/etc/passwd 通常存在)
bash check.sh /nonexistent              # 应输出不存在
文件 '/etc/passwd' 存在且是普通文件。
文件 '/nonexistent' 不存在或不是普通文件。
yy@yy:~$
```

图 20: Enter Caption

任务：创建脚本 `check.sh`，接收文件名参数，用 `test -f` 判断并输出提示。

环境准备：创建脚本文件。

```
nano check.sh
```

写入：

```
#!/bin/bash
if [ -f "$1" ]; then
    echo "文件 '$1' 存在且是普通文件。"
else
    echo "文件 '$1' 不存在或不是普通文件。"
fi
```

保存退出。

步骤：

1. 先用解释器测试（无需执行权限）：

```
bash check.sh /etc/passwd
bash check.sh /nonexistent
```

预期输出：对 /etc/passwd 输出存在；对不存在的输出不存在。

感悟：给 \$1 加双引号防止空格；-f 只检查普通文件。

练习 9：赋予执行权限并运行脚本



```
yy@yy:~$ ls -l check.sh
-rw-rw-r-- 1 yy yy 149  8月 31 14:39 check.sh
yy@yy:~$ ./check.sh /etc/passwd
bash: ./check.sh: Permission denied
yy@yy:~$ chmod +x check.sh
yy@yy:~$ ls -l check.sh
-rwxrwxr-x 1 yy yy 149  8月 31 14:39 check.sh
yy@yy:~$ ./check.sh /etc/passwd
文件 '/etc/passwd' 存在且是普通文件。
```

图 21: Enter Caption

任务：先尝试直接执行（权限拒绝），用 `chmod +x` 赋予权限后再执行，理解原因。

环境准备：已有 `check.sh`。

步骤：

1. 查看权限：

```
ls -l check.sh
```

输出类似 `-rw-r--r--`（无 `x`）。

2. 尝试直接执行：

```
./check.sh /etc/passwd
```

报错 Permission denied。

3. 赋予执行权限：

```
chmod +x check.sh
```

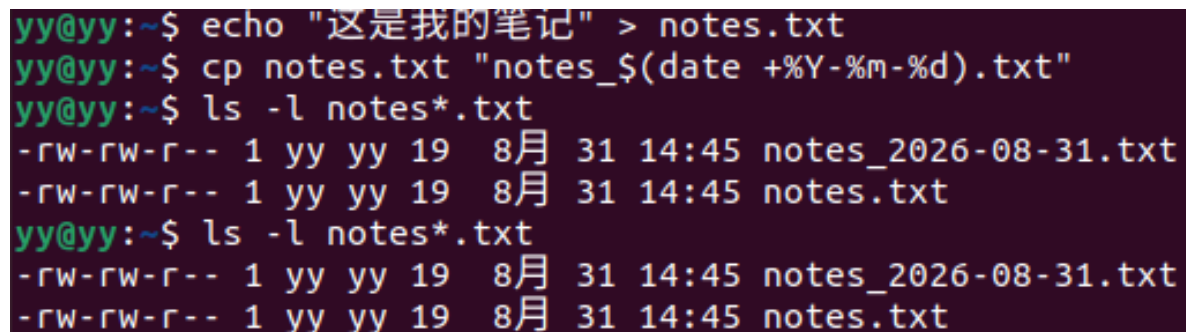
4. 再次查看权限（变为 -rwxr-xr-x）。

5. 再次执行，成功。

预期输出：首次运行报错；chmod 后正常输出。

感悟：UNIX 要求文件有执行位才能直接运行；也可用 `bash check.sh` 绕过。

练习 11：复制文件并添加日期备份名



```
yy@yy:~$ echo "这是我的笔记" > notes.txt
yy@yy:~$ cp notes.txt "notes_$(date +%Y-%m-%d).txt"
yy@yy:~$ ls -l notes*.txt
-rw-rw-r-- 1 yy yy 19  8月 31 14:45 notes_2026-08-31.txt
-rw-rw-r-- 1 yy yy 19  8月 31 14:45 notes.txt
yy@yy:~$ ls -l notes*.txt
-rw-rw-r-- 1 yy yy 19  8月 31 14:45 notes_2026-08-31.txt
-rw-rw-r-- 1 yy yy 19  8月 31 14:45 notes.txt
```

图 22: Enter Caption

任务：将 `notes.txt` 复制为 `notes_YYYY-MM-DD.txt`。

环境准备：创建原始文件。

```
echo "这是我的笔记" > notes.txt
```

步骤：

1. 执行复制：

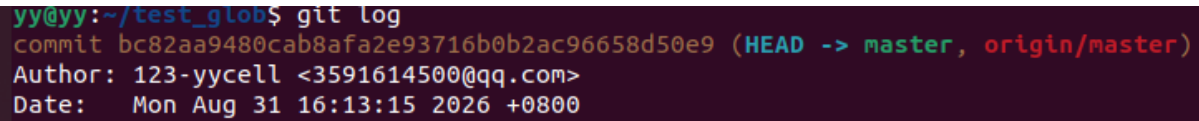
```
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

2. 验证:

```
ls -l notes*.txt
```

预期输出: 生成新文件, 如 `notes_2026-08-31.txt`。

感悟: 命令替换 `$(...)` 动态生成内容; 日期格式 `%Y-%m-%d` 便于排序。



```
yy@yy:~/test_glob$ git log
commit bc82aa9480cab8afa2e93716b0b2ac96658d50e9 (HEAD -> master, origin/master)
Author: 123-yycell <3591614500@qq.com>
Date:   Mon Aug 31 16:13:15 2026 +0800
```

提交第一周实验报告及实验代码目录

图 23: Enter Caption

GitHub 链接: <https://github.com/123yy-cell/-.git>